

Sistemas Distribuídos – Engenharia de Telecomunicações

Comunicação confiável e comunicação orientada a mensagens

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

O que é comunicação confiável? I

Garantir que as mensagens enviadas por um processo cheguem corretamente ao destino, mesmo que ocorram falhas na rede ou nos processos

- Mensagens podem ser perdidas durante a transmissão
- Mensagens podem ser recebidas mais de uma vez
- Mensagens podem chegar em uma ordem diferente da que foram enviadas

O que é comunicação confiável? II

- Se a comunicação usa um canal confiável (ex: TCP), a própria camada de transporte cuida da entrega correta das mensagens
- Se o canal não é confiável (ex: UDP), a aplicação precisa implementar mecanismos extras para garantir a entrega correta

Tipos de falhas em canais e processos I

- **Falhas em canais de comunicação**
 - **Queda:** conexão interrompida (exemplo: cabo desconectado)
 - **Omissão:** mensagem perdida no caminho
 - **Timeout:** tempo de espera esgotado sem resposta
 - **Arbitrária:** comportamento inesperado, como mensagens corrompidas

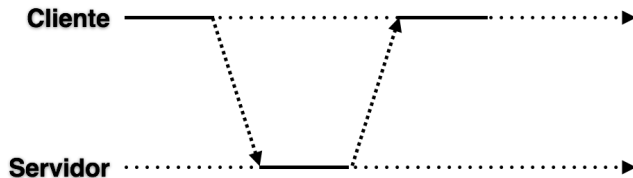
Canais confiáveis (como TCP) conseguem mascarar omissão ou queda usando confirmações (*ack*) e retransmissões

Tipos de falhas em canais e processos II

- **Falhas em processos**
 - **Omissão de envio ou recepção:** processo não envia ou não recebe mensagem
 - **Parada (*crash*):** processo para de funcionar
 - **Arbitrária:** comportamento incorreto, como enviar dados errados

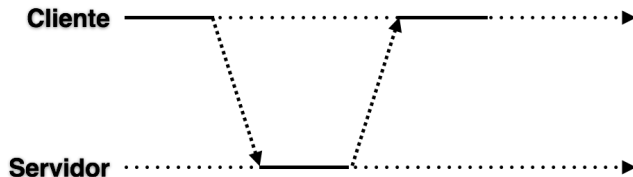
Canais confiáveis (como TCP) não conseguem mascarar falhas por queda do processo, pois o processo não pode responder a mensagens ou enviar confirmações

Modelo cliente-servidor: revisão



- 1 O **cliente** envia um pedido ao **servidor** (exemplo: navegador acessando um site)
- 2 O servidor processa o pedido e envia uma resposta
- 3 O cliente aguarda a resposta do servidor

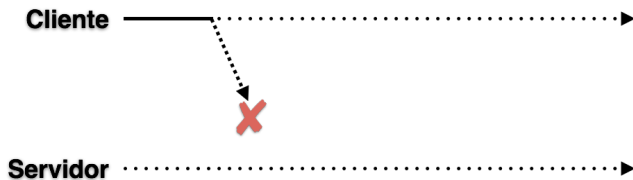
Modelo cliente-servidor: revisão



- 1 O **cliente** envia um pedido ao **servidor** (exemplo: navegador acessando um site)
- 2 O servidor processa o pedido e envia uma resposta
- 3 O cliente aguarda a resposta do servidor

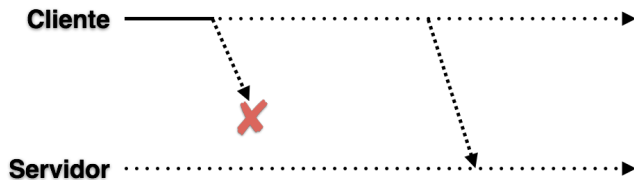
Quais problemas podem acontecer durante essa comunicação?

Problema: servidor não recebe o pedido



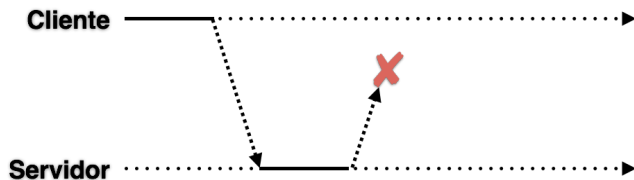
- O que o cliente deve fazer?
 - 1 Detectar a perda usando um tempo limite (*timeout*), como quando um site demora a carregar
 - 2 Reenviar o pedido ao servidor

Problema: servidor não recebe o pedido



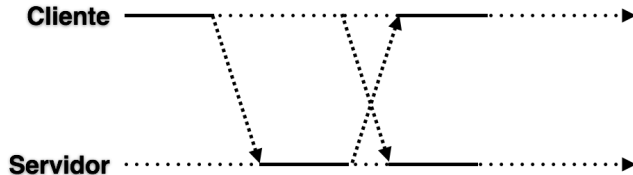
- O que o cliente deve fazer?
 - 1 Detectar a perda usando um tempo limite (*timeout*), como quando um site demora a carregar
 - 2 Reenviar o pedido ao servidor

Problema: resposta não chega ou servidor demora



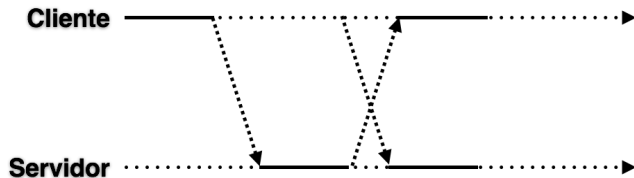
- O que o cliente deve fazer?
 - 1 Esperar até o tempo limite (*timeout*)
 - 2 Se não houver resposta, enviar um novo pedido

Problema: resposta não chega ou servidor demora



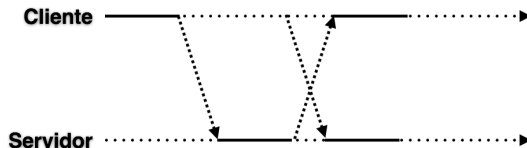
- O que o cliente deve fazer?
 - 1 Esperar até o tempo limite (*timeout*)
 - 2 Se não houver resposta, enviar um novo pedido

Qual o problema de reenviar o pedido?



- O servidor pode tratar cada pedido como novo, sem saber se já processou antes
- Isso pode levar a inconsistências e resultados inesperados, por exemplo, se o pedido for para criar um novo recurso, o servidor pode criar vários recursos idênticos se receber o mesmo pedido várias vezes

Idempotência

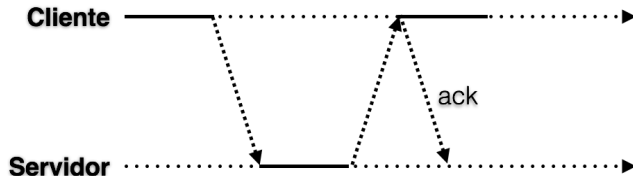


- **Requisições idempotentes:** podem ser repetidas várias vezes e sempre produzem o mesmo resultado. Exemplo: HTTP GET para consultar dados
- **Requisições não idempotentes:** alteram o estado do servidor se repetidas. Exemplo: HTTP POST para criar um novo recurso

Exemplo

Buscar saldo bancário (GET) pode ser repetido sem problemas, mas transferir dinheiro (POST) não!

Como lidar com requisições não idempotentes?



- O servidor precisa manter um histórico dos pedidos já processados e exigir confirmação de entrega (ACK) do cliente
- Cada pedido deve ter um identificador único para evitar duplicidade

Semântica de entrega: quantas vezes a mensagem é processada? I

- **No máximo uma vez** (*at-most-once*): o pedido é processado uma única vez ou não é processado.
- **Ao menos uma vez** (*at-least-once*): o pedido é processado pelo menos uma vez, podendo ser processado mais vezes (duplicado).
- **Exatamente uma vez** (*exactly-once*): o pedido é processado uma única vez, sem duplicidade.

Observação

Em sistemas distribuídos, garantir *exactly-once* é muito difícil. Por isso, normalmente se implementa *at-most-once* ou *at-least-once*.

Semântica de entrega: quantas vezes a mensagem é processada? II

- *At-most-once*: servidor mantém histórico dos pedidos e ignora duplicados (mais complexo, mas confiável).
- *At-least-once*: servidor não mantém histórico, apenas reenvia até receber ACK (mais simples, mas pode duplicar).
- *Exactly-once*: comportamento padrão de chamadas locais, mas difícil em sistemas distribuídos.

Resumo: comunicação confiável

- TCP mascara falhas por omissão (com *ack* e retransmissões), mas não mascara falhas por queda do processo
- Sistemas distribuídos precisam lidar com reenvios, duplicação e falhas silenciosas
- Idempotência é fundamental para garantir a entrega correta das mensagens

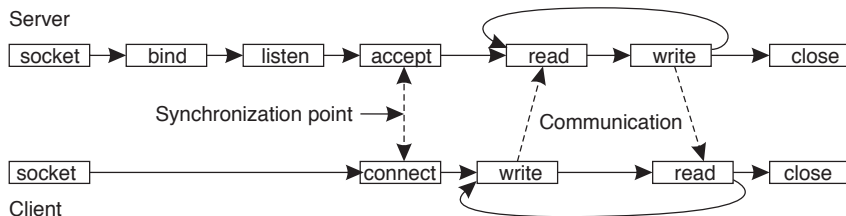
Comunicação orientada a mensagens

Revisão: modelo cliente-servidor

Paradigma pedido e resposta

- **Endereçamento:** como localizar o servidor? Pode ser por conhecimento prévio, *broadcast* ou serviço de nomes
- Comunicação pode ser:
 - **Síncrona:** emissor e receptor ficam bloqueados até enviar ou receber mensagem
 - **Assíncrona:** emissor não fica bloqueado, pode continuar outras tarefas

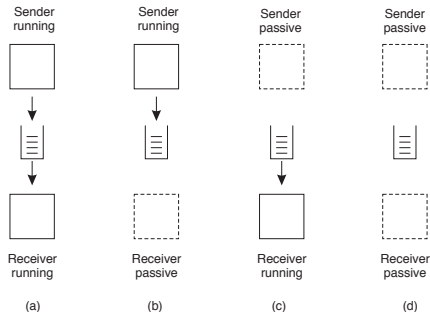
Comunicação transitória: conceito



- Exemplos: RPC e RMI
- Emissor e receptor precisam estar ativos ao mesmo tempo
- As mensagens são trocadas diretamente entre os processos

Comunicação persistente por mensagens

- Em redes WAN, as conexões podem ser instáveis ou cair temporariamente
- Comunicação persistente garante que as mensagens serão entregues mesmo que ocorram falhas temporárias
- Isso é feito usando filas de mensagens: a comunicação é assíncrona e as mensagens ficam armazenadas até o receptor estar disponível



Middleware orientado a mensagens (MOM)

- **Facilita comunicação desacoplada no tempo entre aplicações:** emissor e receptor não precisam estar ativos ao mesmo tempo
- **Garante persistência e roteamento das mensagens:** mensagens podem ser roteadas por servidores intermediários até o destino
- **Permite interoperabilidade entre sistemas diferentes:** aplicações escritas em linguagens ou plataformas distintas podem se comunicar usando filas de mensagens

Comunicação orientada a mensagens

Casos de uso

- **Integração de sistemas heterogêneos:** diferentes linguagens, plataformas ou localizações geográficas
- **E-commerce e sistemas transacionais:** carrinho, pagamento e envio processados por serviços desacoplados
- **Processamento assíncrono:** geração de relatórios, uploads, tarefas em segundo plano
- **Sistemas IoT e Telemetria:** dispositivos enviam dados para servidores via mensagens (exemplo: sensores de temperatura)
- **Pipelines de dados:** extração, transformação e carregamento (ETL), análise em tempo real, integração com Kafka e Spark

Advanced Message Queuing Protocol (AMQP) I

Protocolo Avançado de Enfileiramento de Mensagens

- **AMQP** é um padrão aberto para middleware orientado a mensagens, criado para garantir interoperabilidade entre sistemas diferentes
- Utiliza protocolo binário na camada de aplicação e requer transporte confiável (TCP)
- Oferece autenticação e criptografia para segurança.
- Suporte a vários padrões de comunicação:
 - **Ponto a ponto**: comunicação direta entre emissor e receptor
 - **Publicador-assinante**: um emissor para vários receptores (*publish/subscribe*).
 - **Fila de mensagens**: mensagens podem passar por servidores intermediários
- Oferece três níveis de entrega de mensagens:
 - *at-most-once*, *at-least-once*, *exactly-once*

Advanced Message Queuing Protocol (AMQP) II

Protocolo Avançado de Enfileiramento de Mensagens

- Principais implementações:
 - RabbitMQ – <https://www.rabbitmq.com>
 - Apache Qpid – <https://qpid.apache.org>
 - Apache ActiveMQ – <https://activemq.apache.org>

Message Queue Telemetry Transport (MQTT)

Protocolo de mensagens leve

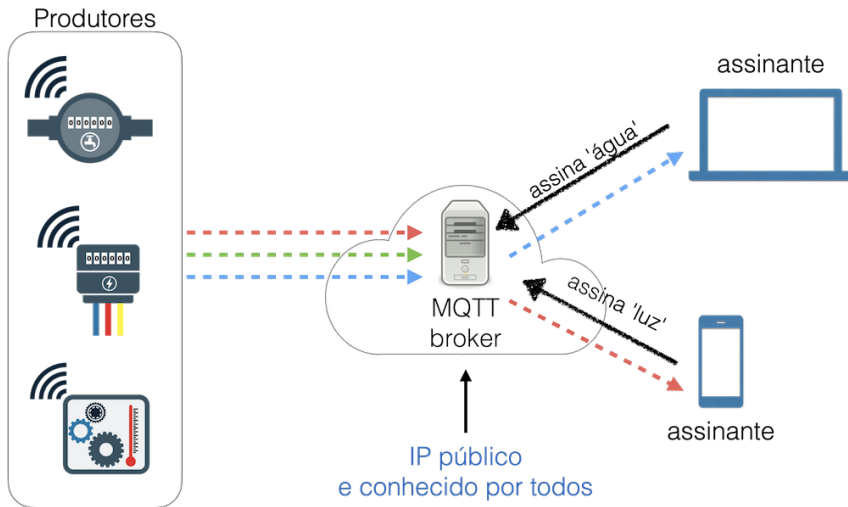
- **MQTT** é um protocolo leve para troca de mensagens, baseado no modelo *publish/subscribe*
- Muito usado em Internet das Coisas (IoT), pois funciona bem em dispositivos com poucos recursos e redes instáveis
- Opera sobre TCP/IP e pode usar TLS para garantir segurança
- Projetado para conexões de longa duração e baixa latência (exemplo: sensores enviando dados continuamente)

Message Queue Telemetry Transport (MQTT)

Garantias de entrega

- **Níveis de Qualidade de Serviço (QoS):**
 - **QoS 0 (At-most-once):** melhor desempenho, mas sem garantias de entrega
 - **QoS 1 (At-least-once):** garante entrega, mas pode haver duplicatas
 - **QoS 2 (Exactly-once):** garante entrega única, porém com maior custo de processamento
- **Segurança:** tem suporte a autenticação do cliente e criptografia das mensagens com TLS

Exemplo: cenário IoT e *broker* MQTT



AMQP vs MQTT

Aspecto	AMQP	MQTT
Modelo de comunicação	<i>Queue/Exchange</i>	<i>Publish/Subscribe</i>
Peso do protocolo	Médio a alto	Leve
Semântica de entrega	at-most, at-least, exactly	at-most, at-least, exactly
Segurança	SASL, TLS	TLS
Persistência de mensagens	Sim	Sim
Destinado à aplicações	Corporativas	IoT, sensores, <i>mobile</i>
Requer rede confiável	Sim	Não

Resumo: AMQP é mais robusto e indicado para aplicações corporativas, enquanto MQTT é leve e ideal para IoT e dispositivos embarcados.

Quando usar AMQP ou MQTT?

- **Use AMQP quando**

- Precisa de roteamento complexo de mensagens
- Requer transações, filas duráveis e controle rigoroso de fluxo
- Atua em ambientes corporativos com serviços robustos

- **Use MQTT quando**

- Está lidando com dispositivos embarcados ou IoT
- Precisa de um protocolo leve e tolerante a falhas de rede
- Deseja simplicidade e comunicação em tempo real (telemetria)

Aulas baseadas em



TANENBAUM, Andrew S; STEEN, Maarten van. **Sistemas Distribuidos: Princípios e paradigmas**. 2. ed.: Prentice Hall, 2008.